

Recursive Checkpoint Strategy

来源：CS336 Assignment 2 – Problem gradient_checkpointing(a) 的完整分析。

核心问题： N 个相同的 Transformer block 堆叠，如何通过嵌套 checkpoint 使 peak activation memory 最小化(忽略 compute cost)?

"Consider a Transformer with N identical blocks stacked sequentially. Without any checkpointing, all N blocks' worth of residuals are kept alive simultaneously, giving $O(N)$ peak activation memory. We have a free hand to wrap any subset of the forward pass in checkpoint, including nesting checkpoint calls inside one another.

(a) What checkpointing strategy minimizes peak activation memory, ignoring the compute cost?

Describe how you would arrange the checkpoint calls (a code sketch is fine), and give the asymptotic peak activation memory and compute of your strategy as a function of N . Assume the residuals saved by a single block dominate any per-checkpoint bookkeeping."

符号标记

- 每个 block 的 residuals 大小为 r (backward 所需的全部中间激活)
- 每个 block 的输入 activation 大小为 x (即一个 checkpoint 保存的内容)
- XL 配置下 $r \approx 3.6$ GiB, $x = 4 \times 2048 \times 2560 \times 4B = 80$ MiB, 即 $r \approx 45x$, 故 $r \gg x$

checkpoint 语义：

1. `checkpoint(f, x)` forward 时只保存输入 x , 丢弃 f 内部所有 residuals
2. backward 进入该 checkpoint 时, 重新跑 f (recompute), 临时物化内部 residuals, 用完即释放
3. 嵌套时: 外层 recompute 会重新执行内层 checkpoint 的 forward (重新产生内层 checkpoint 输入), 这些输入在外层 backward 完成前一直存活

Peak memory 出现在：backward 过程中, “当前活着的所有 checkpoint 输入” + “当前正在 recompute 的那段内部 residuals”之和最大的时刻 (通常在最深叶子的 recompute 时刻)。

$N = 8$ 的完整方案对比

以 $N = 8$ 为例, 逐一推演各方案的 forward/backward 内存:

方案	结构	深度 d	Peak Memory	Compute (fwd 次数)	$r \gg x$ 时
无 checkpoint	8 直连	0	$8r$	1	$8r$
单层 $k = 2$	2 段 $\times$ 4 blocks	1	$2x + 4r$	2	$4r$
单层 $k = 4$	4 段 $\times$ 2 blocks	1	$4x + 2r$	2	$2r$
单层 $k = 8$	8 段 $\times$ 1 block	1	$8x + r$	2	r
二层二叉	$2 \times (2 \times 2)$	2	$3x + 2r$	3	$2r$
三层二叉	$2 \times (2 \times (2 \times 1))$	3	$4x + r$	4	r

计数口径：按“不同数值的 tensor 数”计。嵌套时外层 checkpoint 的输入与内层第一个 checkpoint 的输入是**同一个 tensor 对象**（引用，非拷贝），不重复计。详见下方 $d = 2$ 的完整流程推演。

关键对比：三层二叉 ($4x + r$) vs 单层 $k = 8$ ($8x + r$) —— 同样把 r 的系数压到 1，但嵌套的 x 系数减半；代价是 compute 从 2 涨到 4。

$N = 8, d = 2$ 二层二叉的完整 Forward/Backward 流程

结构定义：

$$\text{cpA}(\underbrace{\text{cpB}(\text{B1}, \text{B2}) \rightarrow \text{cpC}(\text{B3}, \text{B4})}_{\text{左半 H1}}) \rightarrow \text{cpD}(\underbrace{\text{cpE}(\text{B5}, \text{B6}) \rightarrow \text{cpF}(\text{B7}, \text{B8})}_{\text{右半 H2}})$$

各 checkpoint 输入：

- x_0 = 原始输入（cpA、cpB 的输入）
- x_2 = B2 输出 / B3 输入（cpC 的输入）
- x_4 = H1 输出 / H2 输入（cpD 的输入）
- x_6 = B6 输出 / B7 输入（cpF 的输入）

Forward 阶段：嵌套时内层 checkpoint 被外层抑制

时刻	动作	保存/释放	当前存活的 checkpoint (按 tensor 对象)
F1	进入 cpA，保存输入 x_0	$+x$	$\{x_0\}$
F2	cpA 内部：进入 cpB，保存 x_0 (与 cpA 同 tensor)	$+0$	$\{x_0\}$
F3	cpB 内部：跑 B1→B2，不保存任何中间激活	—	$\{x_0\}$
F4	cpB 结束，输出 x_2	—	$\{x_0\}$
F5	cpA 内部：进入 cpC，保存 x_2 (新数值)	$+x$	$\{x_0, x_2\}$
F6	cpC 内部：跑 B3→B4，不保存	—	$\{x_0, x_2\}$
F7	cpC 结束，输出 x_4	—	$\{x_0, x_2\}$
F8	cpA 结束，离开外层作用域，内层保存的 x_2 被释放 (x_0 仍被外层 cpA 持有)	$-x$	$\{x_0\}$
F9	进入 cpD，保存 x_4 (新数值)	$+x$	$\{x_0, x_4\}$
F10	cpD 内部：进入 cpE，保存 x_4 (与 cpD 同 tensor)	$+0$	$\{x_0, x_4\}$
F11	cpE 内部：跑 B5→B6，不保存	—	$\{x_0, x_4\}$
F12	cpE 结束，输出 x_6	—	$\{x_0, x_4\}$
F13	cpD 内部：进入 cpF，保存 x_6 (新数值)	$+x$	$\{x_0, x_4, x_6\}$
F14	cpF 内部：跑 B7→B8，不保存	—	$\{x_0, x_4, x_6\}$
F15	cpF 结束，输出 x_8	—	$\{x_0, x_4, x_6\}$
F16	cpD 结束，离开外层作用域，内层保存的 x_6 被释放 (x_4 仍被外层持有)	$-x$	$\{x_0, x_4\}$

Forward 结束： $\{x_0, x_4\}$ ，共 $2x$ 。

Backward 阶段：从右到左，逐层 recompute

时刻	动作	物化/ 释放	存活 checkpoint (不同数值)	当前 recompute residuals	总内存
B1	从输出 x_8 反向，触发 cpD 的 backward	—	$\{x_0, x_4\}$	—	$2x$
B2	recompute cpD 内部 forward ：重新产生 cpE/cpF 的输入 (x_4 已在手，只新增 x_6)	$+x$	$\{x_0, x_4, x_6\}$	—	$3x$
B3	反向到 cpF，触发其 backward	—	$\{x_0, x_4, x_6\}$	—	$3x$
B4	recompute cpF 内部 forward ：跑 B7→B8，物化全部 residuals	$+2r$	$\{x_0, x_4, x_6\}$	$2r$	$3x + 2r \leftarrow$ Peak
B5	cpF backward 完成，释放 residuals 和 x_6	$-2r, -x$	$\{x_0, x_4\}$	—	$2x$
B6	反向到 cpE，触发其 backward	—	$\{x_0, x_4\}$	—	$2x$
B7	recompute cpE 内部 forward ：跑 B5→B6，物化全部 residuals	$+2r$	$\{x_0, x_4\}$	$2r$	$2x + 2r$
B8	cpE backward 完成，释放 residuals；cpD 的 recompute 上下文随之释放	$-2r$	$\{x_0, x_4\}$	—	$2x$
B9	cpD backward 完成，释放输入 x_4	$-x$	$\{x_0\}$	—	x
B10	反向到 cpA，触发其 backward	—	$\{x_0\}$	—	x

时刻	动作	物化/ 释放	存活 checkpoint (不同数值)	当前 recompute residuals	总内存
B11	recompute cpA 内部 forward : 重新产生 cpB/cpC 的输入 (x_0 已在手, 只新增 x_2)	$+x$	$\{x_0, x_2\}$	—	$2x$
B12	反向到 cpC, 触发其 backward	—	$\{x_0, x_2\}$	—	$2x$
B13	recompute cpC 内部 forward : 跑 B3→B4, 物化全部 residuals	$+2r$	$\{x_0, x_2\}$	$2r$	$2x + 2r$
B14	cpC backward 完成, 释放 residuals 和 x_2	$-2r, -x$	$\{x_0\}$	—	x
B15	反向到 cpB, 触发其 backward	—	$\{x_0\}$	—	x
B16	recompute cpB 内部 forward : 跑 B1→B2, 物化全部 residuals	$+2r$	$\{x_0\}$	$2r$	$x + 2r$
B17	cpB backward 完成, 释放 residuals	$-2r$	$\{x_0\}$	—	x
B18	cpA backward 完成, 释放输入 x_0	$-x$	$\emptyset$	—	0

Peak memory 出现在 B4: $3x + 2r$ (按不同数值的 tensor 计: $\{x_0, x_4, x_6\}$ 三个 checkpoint 加上 cpF recompute 的 $2r$)。

关键机制总结

- 1. **外层抑制内层**: cpA/cpD 结束时, 其内部产生的 checkpoint (如 x_2 、 x_6) 被强制释放, Forward 结束只剩 $2x$
- 2. **同 tensor 去重**: 外层 checkpoint 的输入与内层第一个子 checkpoint 的输入是**同一 tensor 对象** (引用, 非拷贝), 统计内存时只算一份 —— 这就是一般公式中系数是 $1 + (m - 1)d$ 而非 $1 + md$ 的原因

3. **Backward 的 recompute 是递归的**：反向到 cpD 时，先 recompute cpD 的 forward（重新产生 x_6 ），再反向进入 cpF 时，再 recompute cpF 的 forward（物化 B7/B8 的 residuals）
4. **Peak 出现在最深叶子的 recompute 时刻**：B4 时， $\{x_0, x_4, x_6\}$ 三个 checkpoint 同时存活，加上 cpF 的 $2r$ ，得 $3x + 2r$

Compute 递推关系

设 $F(d)$ 为深度 d 嵌套下完成整个流程(forward+backward梯度反向传播)所需的总计算次数(单位:forward)

$$F(0) = 1, \quad F(d) = F(d - 1) + 1$$

每多一层嵌套，backward 时每下潜一层需多**完整recompute所有的block**一次。解得：

$$F(d) = d + 1$$

一般 m 叉树的 Peak 公式

设 d 层完全 m 叉树， $N = m^d$ ，最外层包一个根 checkpoint。在 backward 到最深叶子时，按“不同数值的 tensor 数”统计存活的 checkpoint：

- 根节点：1 个 (x_0)
- 每往下潜一层，新增 $m - 1$ 个不同数值（该层的 m 个兄弟中，路径上第一个的输入与外层输入同 tensor，不重复计；其余 $m - 1$ 个兄弟的输出都是新数值）
- 跨 d 层，共 $1 + (m - 1)d$ 个不同数值的 checkpoint

$$\text{Peak}(d, m) = [1 + (m - 1) d] x + \frac{N}{m^d} r$$

验证 ($N = 8$):

d	m	公式	结果
1	8 (无根, 单层 $k = 8$)	$8x + \frac{8}{8}r$	$8x + r$
2	2	$[1 + 1 \cdot 2]x + \frac{8}{4}r$	$3x + 2r$
3	2	$[1 + 1 \cdot 3]x + \frac{8}{8}r$	$4x + r$

为什么二叉是工程最优选择

Step 1: 固定深度 d , 优化分支数 m

$$f(m) = (m - 1) d x + \frac{Nr}{m^d}$$

对 m 求导并令其为零:

$$\frac{\partial f}{\partial m} = d x - \frac{d N r}{m^{d+1}} = 0 \implies m^{d+1} = \frac{N r}{x} \implies m^* = \left(\frac{N r}{x} \right)^{\frac{1}{d+1}}$$

代回得:

$$f(d) = \left[\left(\frac{N r}{x} \right)^{\frac{1}{d+1}} - 1 \right] d x + (N r)^{\frac{1}{d+1}} x^{\frac{1}{d+1}}$$

当 $A := N r / x \gg 1$ 时, $\left(\frac{N r}{x} \right)^{\frac{1}{d+1}} \gg 1$, 可近似为:

$$f(d) \approx (d + 1) x^{\frac{d}{d+1}} (N r)^{\frac{1}{d+1}}$$

Step 2: 对深度 d 优化

从 Step 1 的结果出发, 最优 m 代回后 peak 只是 d 的函数:

$$f(d) = (d + 1) x^{\frac{d}{d+1}} (N r)^{\frac{1}{d+1}}$$

(i) 换元化简: 令 $t = d + 1$ (则 $\frac{d}{d+1} = 1 - \frac{1}{t}$):

$$f = t \cdot x^{1 - \frac{1}{t}} \cdot (N r)^{\frac{1}{t}} = t \cdot x \cdot \underbrace{x^{-\frac{1}{t}} \cdot (N r)^{\frac{1}{t}}}_{= (N r / x)^{1/t}} = t \cdot x \cdot A^{1/t}, \quad A := \frac{N r}{x}$$

(ii) 取对数: $A^{1/t}$ 不好直接求导, 而 $\ln$ 单调、不改变极值点位置, 故对 $\ln f$ 求导:

$$\ln f = \ln t + \ln x + \frac{\ln A}{t}$$

(iii) 求导并令其为零:

$$\frac{d \ln f}{dt} = \underbrace{\frac{1}{t}}_{\ln t} + \underbrace{0}_{\ln x \text{ 常数}} + \underbrace{\ln A \cdot \left(-\frac{1}{t^2} \right)}_{\frac{\ln A}{t}} = \frac{1}{t} - \frac{\ln A}{t^2} = 0$$

两边同乘 t^2 得 $t - \ln A = 0$, 即 (精确解, 无近似):

$$t^* = \ln A \implies d^* = \ln \frac{Nr}{x} - 1$$

(iv) 最后一步才是近似: 展开 $\ln \frac{Nr}{x} = \ln N + \ln \frac{r}{x}$, 当 N 较大使 $\ln N$ 主导时 ($\ln \frac{r}{x}$ 只是常数偏移, XL 下 $\ln 45 \approx 3.8$):

$$d^* \approx \ln N \implies d^* = O(\log N)$$

(v) 直觉检验 —— 为什么 $t^* = \ln A$ 合理:

$f(t) =$

$\underbrace{t \cdot x}_{\text{checkpoint 存储 (随 } t \text{ 线性涨)}}$

$\cdot$

$\underbrace{A^{1/t}}_{\text{段长压缩 (随 } t \text{ 指数降, 边际收益递减)}}$

极值点正是“多存一份 checkpoint 的代价”与“段长进一步压缩的收益”的平衡点。且 $t^* = \ln A$ 时 $A^{1/t^*} = A^{1/\ln A} = e$, 即每层最优压缩比恰为 e —— 与 Step 3 的 $m^* = e$ 互为两面, 自洽。

(vi) 数值 sanity check (XL, $N = 32$, $r/x \approx 45$):

$$d^* = \ln(32 \times 45) - 1 = \ln 1440 - 1 \approx 6.3$$

而二叉嵌套最多切到 $d = \log_2 32 = 5$ 层 (此时每段已是单 block), $d^* > 5$ 说明 $N = 32$ 时“完全二叉切到底”就是最优; 只有当 N 大到 $\ln(Nr/x) - 1 < \log_2 N$ 时, 才存在“不必切到底”的中间最优深度。

Step 3: 最优分支数与最优 Peak

$$m^* = A^{1/t^*} = \left(\frac{Nr}{x}\right)^{\frac{1}{\ln(Nr/x)}} = e$$
$$f^* = t^* \cdot x \cdot e^{\ln A/t^*} = e x \ln \frac{Nr}{x}$$

结论: 数学上最优分支数 $m^* = e \approx 2.718$, 最近的整数是 $m = 2$ 或 $m = 3$ 。二叉 ($m = 2$) 实现最简单且 N 通常为 2 的幂, 是工程最优选择。

数值验证 ($N = 64$, $r \gg x$, 比较 $m \cdot d$ 系数):

m	$d = \log_m 64$	$m \cdot d$
2	6	12
3	≈ 3.79	≈ 11.4

m	$d = \log_m 64$	$m \cdot d$
4	3	12
8	2	16

$m \approx e$ 附近（2、3、4）都接近最优，远离 e 的 m （如 8）明显变差 —— 印证了理论。

最终渐近结论

量	公式	阶
最优深度	$d^* \approx \ln \frac{Nr}{x}$	$O(\log N)$
Peak memory	$\text{Peak}^* = e x \ln \frac{Nr}{x}$	$O(\log N)$
Compute（forward 次数）	$F = d^* + 1$	$O(\log N)$

即 recursive binary checkpointing 把 peak activation memory 从不 checkpoint 的 $O(N)$ 压到 $O(\log N)$ ，代价仅为 $O(\log N)$ 倍的 forward 重算 —— 这就是Problem (a)的最优策略。

参考实现

```

from torch.utils.checkpoint import checkpoint

def C(f, *args):
    return checkpoint(f, *args, use_reentrant=False)

def run(blocks: list[Block], x: Tensor):
    if len(blocks) == 1:
        return C(lambda x: blocks[0](x), x)
    mid = len(blocks) // 2
    left, right = blocks[:mid], blocks[mid:]
    x = C(lambda x: run(left, x), x)
    x = C(lambda x: run(right, x), x)
    return x

y = run(blocks, x)

```

关键设计决策：

决策点	选择	理由
use_reentrant=False	新版实现	只保存输入引用（同 tensor 去重），不复制；旧版 True 会多存一份，已 deprecated
叶子也 checkpoint	是	$r \gg x$ 时把 r 系数压到 1，极致省内存；若 $r \approx x$ 可改为叶子直连省 overhead
二分切分	完全二叉	$m^* = e \approx 2.718$ ，整数最优池；递归实现最简洁

内存-计算权衡验证： $N = 8$ 时 $\text{peak} = 4x + r$ ($d = 3$ 层)， $\text{compute} = 4$ 次 forward —— 与理论一致。